RESEARCH OVERVIEW

How to Reliably Evaluate Agent-Generated Performance Optimizations

Junming Liang · Anhui Normal University · July 2026 – present

1. Background and Problem

Application. The core programs of quantitative trading systems—market-data handling, factor computation, order and position processing—are mostly written in C++ or Rust and are highly sensitive to latency and throughput; engineers routinely iterate for gains of a few percent. They must also be strictly correct: a subtle change in logic can directly affect orders and positions, while existing tests usually cover only typical inputs, so passing them does not mean nothing is broken. Optimization here means being faster without being wrong.

Workflow. I carry out such optimizations with multiple coding agents: a profiler such as `perf` locates the hotspots; an agent, given the hotspot, the relevant code and the input/output requirements, proposes a candidate patch; and the patch is compiled, checked, measured, and then kept, repaired or discarded. Agents can propose many candidates in a short time, so the bottleneck becomes the decision: is a patch a genuine improvement, and is it worth accepting?

Why this decision is hard. Low-latency developers I talk with agree that optimization presupposes an accurate evaluation standard, and that this standard is the hardest part to build:

- **Measurements are noisy.** On my workload, repeated runs of the same program on the same machine vary with a standard deviation of 7–10% of the mean without CPU pinning (about 5% with pinning on the same day), while many optimizations gain only a few percent. Environment isolation—CPU pinning, process isolation, real-time scheduling—reduces the noise, and production machines for low-latency trading are typically configured this way; shared cloud and CI environments often cannot be isolated. Either way, the noise does not disappear.
- **Estimates can be biased**, wrong in one direction however many runs are made. A candidate that always runs after the baseline may benefit from warm caches; concurrent and sequential runs can give different gains; and a test environment unlike production can mislead. Selection adds more: among 20 useless patches, one or two will look faster by chance, and stopping at the first significant result pushes the error rate far above the nominal 5%.
- **The budget is limited.** Every extra minute spent on one candidate is a minute not spent on another; the more candidates, the sharper this trade-off.

Research question. *When the evaluation budget is limited, what evidence is sufficient to accept an agent-generated performance optimization?* The problem is not unique to my work. auto_optimization, a public project inspired by Karpathy's autoresearch, lets an agent optimize an order-book program, keeping any change that measures faster than the current best and passes the tests. Five of the seven changes it kept gained only 0.2–0.8% each; its benchmark script configures environment isolation, but no measurement variability is reported, so the records alone cannot confirm these small gains (Figure 1). Chen et al. (2026) found that many official reference patches in published agent performance benchmarks, replayed on other machines, do not consistently meet the benchmarks' own validity criteria.

Goal. I treat the two risks differently. **Correctness is an absolute veto:** a candidate that fails any check is rejected or sent back for repair, and speed never buys it back. **Performance is a statistical judgment:** control how often changes without real benefit are accepted, attach an uncertainty to real gains, and treat "not confirmed within this budget" as an explicit outcome.

2. The Evaluation Pipeline

The pipeline decides whether an agent's performance patch is adopted. A patch is timed only after it passes correctness checks, and rules fixed before measurement then judge whether the speedup is real and large enough. Every patch ends as accepted, rejected (or sent back for repair), or not confirmed within the budget. The agent that wrote the patch has no say in the verdict, and all raw records are kept for independent recomputation.

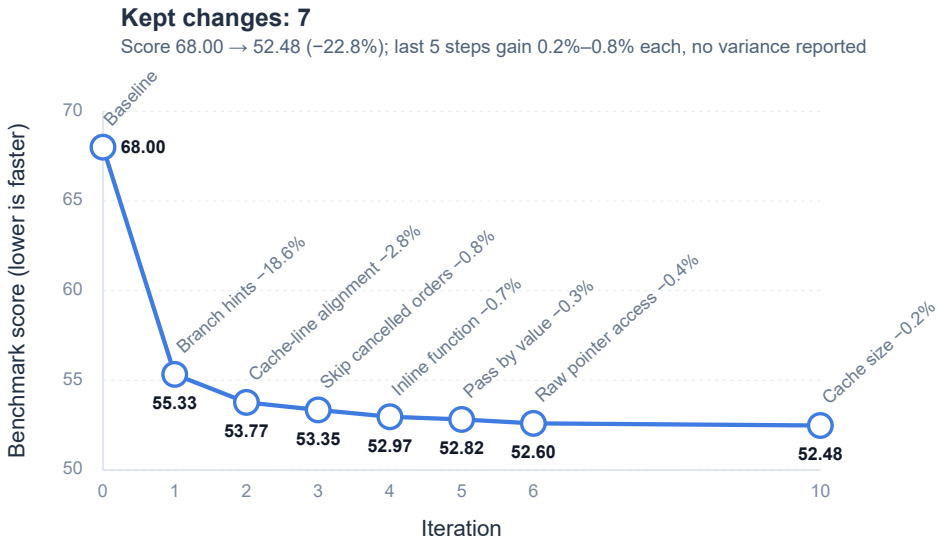


Figure 1. The public auto_optimization loop. A change is kept if it beats the current best and passes 14 unit tests; in 30 minutes the agent ran 20 iterations and kept 7, discarding the rest (slower, failed builds or crashes). The benchmark script configures CPU pinning, process isolation and real-time scheduling; each point is the median of 40 runs. Score: mean time for a batch of about 36,000 order messages, in units of 10 µs.

Step 1: Correctness first. On the test workload, the candidate's final program state must match the original's exactly. Because the workload holds only typical inputs, I also (1) write an *input contract* stating which inputs must be handled correctly and how malformed input is treated; (2) run differential tests on boundary and malformed inputs against the original program, judging inputs on which the original's behavior is undefined by the contract; (3) run ASan and UBSan; and (4) have an agent other than the proposer review the patch and the compiler warnings. A failing patch is rejected or sent back however fast it is; a repaired patch is re-checked and re-measured as a new candidate. Every check and timing run first verifies the patch's hash, so a stale version cannot slip in.

Step 2: Is the speedup real and large enough?

- **Paired, windowed measurement.** The original and the candidate run back to back as a pair, in random order, and each pair's time difference is one measurement. Consecutive pairs form windows in which both orders appear equally often, to balance run-order effects within each window; the example's confirmation used 10 windows of 58 pairs, sized from the time budget and per-pair cost.
- **Window-level interval.** A 95% t-interval is computed from the window means (assumed approximately independent), because noise drifts over time and treating hundreds of pairs as independent would overstate confidence.
- **Two-level conclusion.** An interval wholly above zero shows a gain; wholly above a pre-set minimum effect (1% of the original's time by default, adjustable in advance), the gain is worth keeping. Two cross-checks, the plain mean and the number of pairs won, must agree or be explained.

Step 3: Prevent biased estimates.

- **Run order** is randomized and balanced; in one batch, the two orders gave gains about 57 ms apart.
- **The measurement environment follows deployment.** Some gains depend on the environment—a change that reduces cache misses may gain differently on a dedicated core than on a shared one—and in the wrong environment more runs only measure the wrong thing more precisely. So measurement is pinned and isolated if production is, and runs under the same shared conditions if the program is deployed on cloud or CI machines.
- **Selection.** Candidates that pass exploration tend to be the lucky ones, so exploration only selects; the verdict uses fresh confirmation data alone.
- **Analysis flexibility.** The metric, the threshold and the sample size are fixed before measurement, with no changes or early stopping afterwards.

Step 4: Decide within a budget. Each candidate gets a fixed time—about 10 minutes of exploration and 30 of confirmation—and only those whose exploration estimate reaches the minimum effect go on

to confirmation. The detection capability is estimated during planning: in the example, a change of about 1.5–2% is estimated to have roughly an 80% chance of being detected (interval above zero); a smaller true gain will likely end as "not confirmed within this budget" rather than trigger open-ended re-measurement. Every verdict reports its machine time.

3. A Continuous Optimization Line

The pipeline above decides one candidate. In practice the agent keeps going: once a patch lands, the hotspot moves, and the next candidate has to be judged against the new version, not the original. A continuous line runs this loop under one authorization written before the first run, and ends with one chart of every landed step.

Opening authorization. Before the first candidate, the line fixes once: the model that writes the patches; the total budget, the candidate cap and the part reserved for the final re-measure; the allowed changes and what stays frozen (data layout, threads and cores, compiler flags, workload, the timing boundary); the screening and confirmation rules, the same for every candidate; and the stop rule. Within this authorization the agent does not stop to ask. A later change from me is appended with its date and applies only to candidates opened after it.

One candidate at a time, against the landed version. Each candidate passes the correctness checks of Step 1, then a short screen, then a fresh confirmation against the version landed last. A confirmed candidate lands and becomes the baseline for the next; any other candidate ends with its cost recorded, and the same idea may return later only as a new candidate that inherits no evidence. After each landing the program is profiled again, because the hotspot moves, and candidates screened against the previous version are screened again or dropped.

Two floors in place of hardware counters. Virtual machines often expose no hardware performance counters, so IPC and cache-miss readings are missing. Two instrument builds of the landed version stand in for them: a *compute floor* does the same work per item on data shrunk to fit in cache, and a *data floor* keeps the same loads and stores but removes the computation. Measured together with the real version in one session, they show whether the time is mostly computation or mostly waiting for data, and how far the current design is from its reference. They are instruments, never candidates, and they do not bound designs that change the layout or the bytes touched.

A ledger on disk. After every step the agent rewrites a ledger: the authorization, the acceptance of the measurement, the floors of each landed version, the landed chain, the budget spent by stage, the directions tried grouped by cost type, and the next step. After a context reset or in a new session the agent resumes from the ledger, never from recollection; an incomplete run is rerun as a new attempt counted in the budget.

Stop, then re-measure together. The line stops when the budget is spent, when N consecutive hypotheses fail to land, or when the candidate cap is reached, whichever comes first. Then the original and every landed version are re-measured in one session, interleaved on the same host, and the chart is drawn from those numbers alone, one metric per chart. Confirmation numbers from different sessions are never joined into one curve; only landed, confirmed optimizations become points; and a step that does not reproduce stays on the chart and is reported.

4. Evidence: A Single Optimization on a Public Order Book

Outcome. The example is a public C++ limit-order-book program that simulates an exchange's order book and matching engine; the test workload is 600,000 order instructions (new orders, cancels, modifications, market orders, etc.). The agent's patch replaced heavyweight C++ stream parsing and formatting with `std::from_chars` and `std::to_chars`. **The verdict was accept:** the mean time to process all instructions fell from 1.83 s to 1.28 s, **a 30.0% reduction** (550.6 ms; 95% confidence interval 527.4–573.7 ms), and the program state at the end of every timed run matched the original's. This is the time for the whole batch, not per-order latency.

Correctness: output matching is not enough. An earlier version of this patch left the final program state identical to the original's on all 600,000 test instructions, yet it had three defects the test data never exercises: it dropped inputs the original accepts (e.g., lines with leading spaces), it could read uninitialized values on malformed lines, and its output buffer relied on an unstated capacity assumption, exposed by a compiler warning. Output matching on the test data cannot catch such

defects in principle. An independent review found them, and they were then turned into differential tests. The final version passed 28 differential test cases, including leading spaces, numeric boundaries and malformed input, and the sanitizers found no problem introduced by the patch.

Performance: confirmed with fresh data. Exploration (192 pairs) estimated a gain of about 566 ms, above the 1% threshold; confirmation (580 fresh pairs) gave the result above, with the interval's lower bound far above the threshold (18.3 ms). Both cross-checks agreed; the candidate was faster in 577 of 580 pairs. The two stages took about 43 minutes of machine time, including builds and checks, within the pre-set limits.

5. Evidence: A Continuous Line on a Market-Data Callback

Setting. The target is the per-packet callback of an in-house market-data receiver written in C++: it converts each feed record into the internal layout and writes it into a shared ring, from which a writer thread saves it to files. The metric is the callback's time per packet, stamped on a monotonic clock at entry and exit. Each run replays about 200,000 packets from a simulated feed (snapshots, orders and trades from two exchanges) on a virtual machine (Xeon 8173M) without hardware counters; it yields two numbers, the median and the p99 of per-packet time, after the first 65,536 packets are discarded as warmup. Only the callback path and its startup precomputation could change; the writer, logging and clock threads, the ring layout, the core assignment and the compiler flags were frozen.

Rules fixed before the first run. Budget: 4 hours of machine time, 45 minutes of it reserved for the final re-measure; at most 10 hypotheses. Acceptance: the same build run as both arms for 6 pairs showed no significant difference in either metric, and the correctness gate rejected a deliberately broken build that copies a security code one character short. Correctness for every candidate: the four output files byte-identical to the baseline's once the two receive-time columns are masked, plus unit tests; before landing, also a whole-program simulation, ASan with UBSan, and TSan. Screening: 3 pairs; a candidate goes on if either metric's mean difference favors it. Confirmation: 10 fresh pairs in alternating order, with a two-sided exact sign-flip test on each metric (smallest attainable $p = 2/1024$, about 0.002). A candidate lands if one metric improves with $p < 0.025$ and the other is not significantly worse at 0.05. Unlike Step 2, each run already summarizes more than 100,000 packets, so the run pair is the unit, and no minimum effect was set: the line asks only whether each step is real. Stop: budget spent, 3 consecutive hypotheses without a landing, or the cap. A floor-distance condition was not used, because the allowed changes could alter the bytes each record touches.

Five of six hypotheses landed. Confirmed against the version before each:

- prefetching all ring slots of a snapshot packet before writing them: p99 from 1274.5 to 691.5 ns;
- zeroing small records with unrolled 16-byte stores instead of `rep stos`: median from 120.7 to 104.2 ns;
- dividing snapshot prices two at a time with `divpd`, bit-identical to dividing one by one: p99 from 669.2 to 633.2 ns;
- finding the end of a security code with SSE2 byte compares: median from 103.2 to 96.7 ns;
- inlining the fast path of two diagnostic checks and marking the slow path cold: median from 98.5 to 93.6 ns.

Hidden symbol visibility for the per-record functions failed its screen. Part of the fifth hypothesis, a shortcut that skips a per-thread check, was withdrawn before measurement: a new thread can reuse the handle of a thread that has exited, and the shortcut would then miss it. Three times the gate stopped a candidate because its new test code was wrong (a misaligned reference flagged by UBSan; twice, test samples drawn from the short-period low bits of a linear congruential generator, which hit too few cases); each time the test was fixed and the candidate re-run from the start as a new candidate, and the product code always matched the baseline's output. The line stopped on budget: after the sixth hypothesis, at about 183 minutes, only about 12 minutes were left for candidates, and one candidate takes at least 13. The final re-measure took 9.6 of its reserved 45 minutes, so the reserve was overestimated.

Convergence. The final re-measure ran 12 versions in one session, 10 runs each in rotating order, each built with its own default flags: the starting point, six optimizations landed on this program earlier (each accepted at the time under its own, earlier criterion), and this line's five. The median

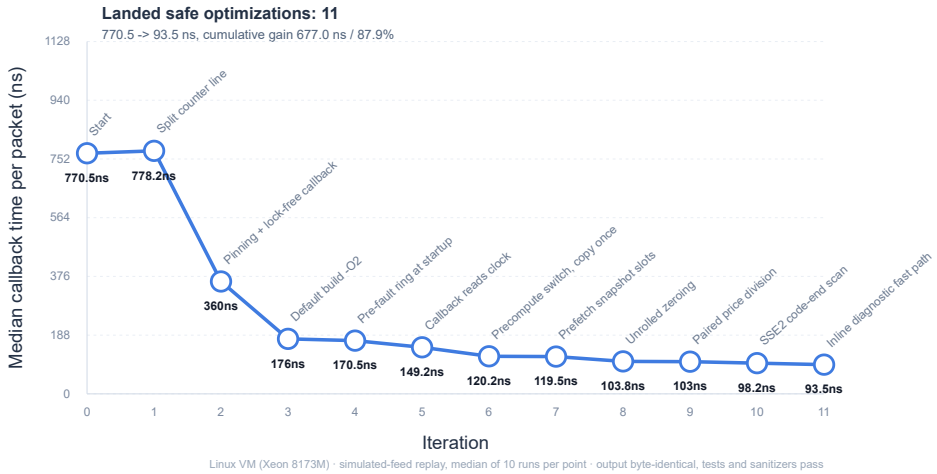


Figure 2. Median per-packet callback time of every landed version, re-measured together; each point is the median of 10 runs. Points 1–6 were landed before the line, points 7–11 by it. Point 1 did not reproduce in whole-program replay and stays on the chart.

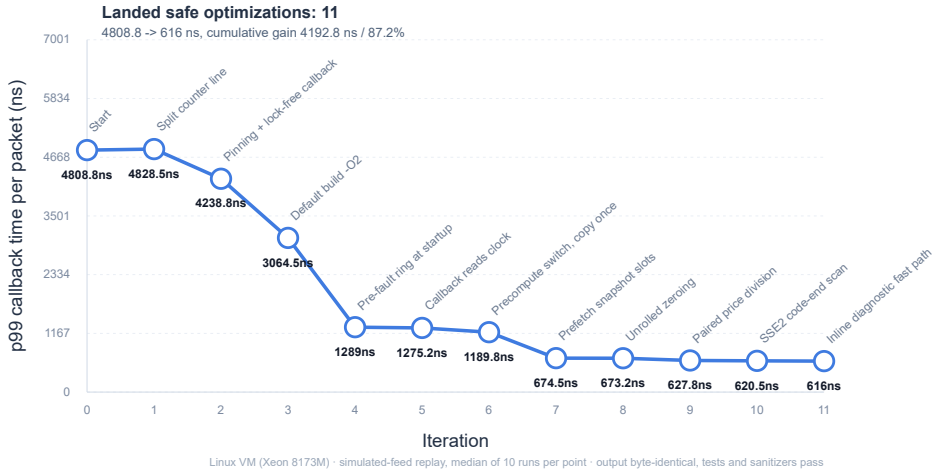


Figure 3. The same versions and runs, p99 of per-packet callback time. Points 1 and 10 did not reproduce on this metric and stay on the chart.

fell from 770.5 to 93.5 ns (−87.9%) and the p99 from 4808.8 to 616 ns (−87.2%); this line alone took them from 120.2 to 93.5 ns and from 1189.8 to 616 ns. Two gains did not reproduce and stay on the charts: moving the ring counters onto their own cache line, which had been measured on the function alone, made no measurable difference in whole-program replay, and the p99 gain of the SSE2 step did not reproduce, although its median gain, which also qualified it, did. In the last floor session the final version's median, 92.6 ns, was 1.5 ns above its compute floor (91.1 ns), and its p99, 615 ns, was no higher than its data floor (640 ns). Those floors were measured on the frozen ring layout and bytes touched, with the callback on one core. What remains is mostly frozen cost, such as one clock read per record (about 22 ns) and the atomic claim of a ring slot, or work outside the allowed scope, such as the cold writes into the ring.

6. Contributions and Next Steps

Contributions. I designed and implemented a pipeline for evaluating agent-generated optimizations in which correctness is an absolute veto and performance must be confirmed with fresh data within a pre-declared budget. There are only three outcomes—accept, reject (or send back for repair), and not confirmed within this budget—each reported with its cost. The order-book case shows that output matching plus timing would let through a patch that matches the original exactly on the test data yet is defective; the problems found by independent review are now differential tests, and the final version, confirmed on fresh data, still cut the batch time substantially. For agents that keep optimizing, I extended the pipeline into a continuous line: one authorization written in advance, each candidate confirmed against the version landed last, two floors that separate computation from waiting for data without hardware counters, a ledger on disk, and a final re-measure of all landed versions in one session from which the convergence chart is drawn. On a market-data callback the line landed five of six hypotheses, and with the program's earlier steps the median per-packet time fell by 87.9%; the same re-measure showed that two earlier gains did not reproduce. The statistical methods are established; my focus is how evidence should be gathered, and verdicts made, when agents generate candidates at scale.

Open questions and next steps.

- **Correctness.** Whether the checks are complete is untested. Next: fuzz the differential-test inputs, and inject known bugs to measure missed bugs and wrongly rejected valid patches.
- **False acceptance.** How often the pipeline accepts a change with no real benefit is untested. Next: count how often comment-only patches that leave the compiled result unchanged are judged faster.
- **Budget.** A 30% gain is far above the 1.5–2% this budget can resolve, so the budget went untested. Next: slow the program by a known 1–5% to find the smallest gain a given budget confirms, on pinned machines and in cloud and CI.
- **Continuous line.** Each confirmation controls its own error rate, but how often a line of many candidates lands at least one change with no real benefit is untested, and the line set no minimum effect. Next: feed no-op candidates through a line to count false landings, and run a line on a public program so that it can be replicated.
- **Bias.** On the final version, a first comparison with concurrent runs was inconclusive, and other days are untested. Next: repeat confirmation across days and with concurrent runs; when several candidates compete, confirm the winner on fresh data, with a threshold that tightens as their number grows.

Longer term, I want to study whether feeding this evidence back to the agents improves their patches.